Network Diagnostics Suite - Documento técnico

Aplicación Android en React Native y TypeScript para diagnóstico de redes y registro de instalaciones. Utiliza módulos Expo compilados en el APK, un módulo Kotlin para comunicaciones, WatermelonDB para persistencia y un backend Node de sincronización.

1. Arquitectura

Capa	Implementación
Interfaz	React Native, TypeScript y componentes de cámara y formularios
Descubrimiento	NsdManager para mDNS y sockets TCP/UDP de Android
Protocolos	Cliente BER/SNMP propio y SSH embebido con JSch
Evidencia	Camera, Location y FileSystem compilados en el APK
Persistencia	WatermelonDB 0.28 sobre SQLite
Sincronización	Outbox persistente, NetInfo, reintentos y versiones
Reportes	HTML, Expo Print y visor Android PdfRenderer

Las comunicaciones SNMP y SSH salen directamente del dispositivo Android. HTTP se utiliza para catálogo, entidades y adjuntos. El servidor no recibe las contraseñas SSH ni ejecuta diagnósticos de red en nombre del teléfono.

2. Descubrimiento de dispositivos

Se admiten subredes IPv4 /22 a /32, hasta 1024 hosts. mDNS busca servicios SSH, HTTP, HTTPS, workstation y SNMP durante una ventana acotada. El sondeo TCP prueba los puertos 22, 80, 443 y 8291; si no hubo conexión, envía una consulta al puerto SNMP configurado.

Un equipo se incluye cuando acepta una conexión o responde un datagrama. No se considera activo únicamente por un rechazo de conexión, que el NAT de un emulador puede sintetizar. La app muestra progreso y permite cancelar. La MAC se obtiene cuando Android permite leerla; hostname y nombre de servicio se muestran si fueron anunciados. No se completan los datos ausentes con valores inventados.

Los equipos tienen un ID persistente independiente de la IP. La coincidencia exige el mismo sitio y compara MAC si ambos registros la tienen; en otro caso utiliza IP dentro de ese sitio. Esto separa los historiales de sitios que reutilizan direcciones privadas.

3. Subset SNMP

El cliente de `src/protocol/snmp.ts` implementa GetRequest, GetNextRequest y GetResponse de SNMPv1/v2c. Construye y analiza BER/ASN.1 sin delegar el protocolo a una biblioteca SNMP. El transporte usa UDP/161 por defecto, con puerto configurable, timeout de 100 a 15000 ms y un reintento. El socket conectado filtra el host y puerto de origen.

El parser admite SEQUENCE, INTEGER con signo, OCTET STRING, NULL, OID base-128, IpAddress, Counter32, Gauge32, TimeTicks y Counter64 como cadena decimal. Soporta longitudes definidas cortas y largas. Rechaza truncamientos, longitudes indefinidas y bytes adicionales. Las vistas binarias se normalizan a Buffer antes de decodificar texto.

La respuesta debe corresponder al request-id, versión, community, PDU y OIDs solicitados. GETNEXT exige avance lexicográfico y finaliza ante fin de MIB o noSuchName en v1. El recorrido obtiene los índices reales de hasta 64 interfaces, incluidos índices no consecutivos.

Dato	OID
sysDescr.0	1.3.6.1.2.1.1.1.0
sysUpTime.0	1.3.6.1.2.1.1.3.0
sysName.0	1.3.6.1.2.1.1.5.0
ifIndex	1.3.6.1.2.1.2.2.1.1
ifDescr	1.3.6.1.2.1.2.2.1.2
ifInOctets	1.3.6.1.2.1.2.2.1.10
ifOutOctets	1.3.6.1.2.1.2.2.1.16

Los contadores de octetos son acumulados de 32 bits, con vuelta al alcanzar 2^32; no representan una velocidad instantánea. No se implementan SET, traps, GetBulk, SNMPv3 ni MIBs de fabricantes. SNMPv1/v2c transmite la community sin cifrar y se limita al laboratorio.

4. SSH y credenciales

El módulo Android utiliza JSch. Antes de autenticar, obtiene la clave pública del host sin enviar credenciales. La interfaz muestra su huella SHA-256 y solicita contrastarla antes de aceptarla. Durante la conexión autenticada, un HostKeyRepository estricto exige la huella fijada y rechaza cambios.

Los comandos están cerrados por fabricante: Linux/OpenWrt (`uptime`, `ip -s address show`, `ip route show`), MikroTik (`/system resource print`, `/interface print stats`, `/ip route print`) y Cisco (`show version`, `show interfaces`, `show ip route`). Android también valida la lista. No se concatenan entradas de usuario ni se ofrece una shell interactiva. La ejecución tiene un plazo de veinte segundos y un límite de salida de 1 MiB.

Las contraseñas solo se conservan si se activa Recordar de forma segura. SecureStore utiliza el almacenamiento protegido por Android Keystore. Contraseñas, community, token y huellas no se guardan en WatermelonDB ni se incluyen en la outbox. La app permite olvidarlos. La autenticación SSH usa contraseña; no se implementa gestión de claves privadas.

5. Identificación, evidencia y PDF

La cámara reconoce QR, EAN-8/13, Code128/39, PDF417 y DataMatrix. Los contenidos se convierten en identificadores, sin navegar automáticamente a URLs de los códigos. La ficha técnica se obtiene del catálogo del backend y se conserva localmente.

Una instalación requiere equipo, sitio, técnico, notas, ubicación y entre una y seis fotos. Cada foto conserva archivo privado, fecha, coordenadas y precisión. Se solicitan permisos en primer plano, sin micrófono ni ubicación en segundo plano. En Android, el proveedor LocationManager funciona sin Google Play Services. La consulta GPS tiene un plazo de 25 segundos y exige una lectura reciente. El borrador se guarda automáticamente.

La instalación se persiste antes de generar el PDF. Expo Print recibe HTML con notas escapadas, fotos base64, coordenadas y hasta veinte diagnósticos del mismo equipo y sitio. El PDF se guarda en el directorio privado y puede abrirse sin conexión con PdfRenderer o compartirse. Si falla la generación, se conserva el registro para reintentar.

La ubicación simulada se identifica tanto en la app como en el PDF. No hay firma digital ni cifrado integral de la base o de las fotografías.

6. Persistencia y cola offline

WatermelonDB almacena filas con clave compuesta y payload JSON que incluye entidad, ID, versión y fecha. LocalStore serializa escritores. Un writer prepara los cambios y los confirma en un único batch SQLite: la entidad y su acción de outbox se guardan juntas antes de actualizar la interfaz.

Cada edición genera un snapshot inmutable con actionId y createdAt creciente. Un único worker procesa FIFO. Una acción posterior de la misma entidad espera si la anterior presenta conflicto o error. Tras confirmar, se elimina la acción y se propaga la versión remota sin sobrescribir ediciones locales posteriores. Al reiniciar, las acciones interrumpidas en sending vuelven a pending.

Los estados son pending, sending, conflict y failed. Los errores transitorios conservan la acción y aplican backoff de dos segundos a cinco minutos más jitter, sin agotar un número fijo de intentos. Los errores 4xx permanentes, salvo 408/429, requieren revisión. HTTP 409 conserva la versión y evidencia remota: se puede elegir la versión local o la del servidor; antes de reemplazar se conserva un respaldo local.

NetInfo dispara el envío al recuperar conectividad. También se intenta al volver a primer plano y cada veinte segundos con la app activa. Si se informa desconexión no se inicia el envío; las peticiones en curso tienen un timeout de veinte segundos. No se promete sincronización permanente con la app cerrada. Los equipos, instalaciones y diagnósticos siguen disponibles offline.

7. Backend de sincronización

El servidor Node ofrece catálogo, consulta de entidades, descarga de adjuntos, health y POST /v1/sync. Persiste versiones y recibos por actionId. Una acción repetida devuelve el recibo sin duplicar el cambio. Las escrituras se serializan y el estado se confirma mediante un archivo temporal y rename.

Los adjuntos JPEG/PDF se identifican por SHA-256 y se valida su tipo. Una instalación debe incluir foto, ubicación y PDF. El límite de petición es 40 MiB y hasta diez adjuntos. Los payloads con campos de credenciales se rechazan. El token se compara antes de atender las operaciones.

Es un backend mínimo para validar el flujo end-to-end. No incluye administración multiusuario ni controles de producción. La conexión de laboratorio usa HTTP; fuera de ese entorno se requiere HTTPS y autenticación adecuada.

8. Validación y alcance de la demostración

Las pruebas automatizadas verifican codificación BER, correlación, UDP real, identidad de equipos, atomicidad, orden FIFO, reintentos, conflictos, idempotencia y adjuntos del backend.

La demostración ejecuta el APK en Android Emulator y consulta una PC física en la LAN. El agente incluido obtiene interfaces e índices de Windows y contadores mediante Get-NetAdapterStatistics; refresca cada cinco segundos y no responde si la lectura tiene más de treinta segundos. sysUpTime corresponde al tiempo activo del agente y Counter32 se calcula a partir de enteros decimales con BigInt.

En el emulador se verificaron SNMP, SSH contra un servidor local de prueba, persistencia tras reiniciar, captura de evidencia, generación y apertura de PDF offline y envío de adjuntos al reconectar. Cámara y GPS son simulados. No se afirma haber probado un Android físico ni un router físico; el reporte de emulador no acredita una visita de campo.

Referencias

- RFC 1157: https://www.rfc-editor.org/rfc/rfc1157.

- RFC 1213: https://www.rfc-editor.org/rfc/rfc1213.

- WatermelonDB: https://watermelondb.dev/docs.

- JSch: https://github.com/mwiede/jsch.

- Android PdfRenderer: https://developer.android.com/reference/android/graphics/pdf/PdfRenderer.